```
users.filter(id.eq(1)).first::<User>(&conn)?;
```

Connection pooling (deadpool/r2d2): Opening a database connection for every request is expensive. Instead, use a connection pool to reuse the existing connections efficiently:

```
With deadpool (async-friendly):
use deadpool_postgres::Pool;
async fn get_user(pool: Pool) {
    let client = pool.get().await.unwrap();
    let rows = client.query("SELECT * FROM users", &[]).
await.unwrap();
}
```

If you are using a synchronous setup, then *r2d2* is a good option. Connection pool limits help reduce latency and avoid resource exhaustion under load. With Actix, the pool is usually stored in `web::Data`, so it can be shared.

The wildcard: High-speed NoSQL caching*:* Repeated queries can slow things down, even with optimised SQL, where NoSQL caching layers like Redis or ScyllaDB come into play.

Here's an example with Redis:

```
use redis::AsyncCommands;
let mut conn = redis_client.get_async_connection().await?;
let cached: String = conn.get("user:1").await.unwrap_or_
default();
```

You can cache data that is hit frequently and bypass the database entirely for repeat requests. This dramatically reduces response time and database load. A common pattern is:

- Check cache → return if found
- Else query DB → store in cache → return

Advanced performance tuning

Once the API is working, the next step is to squeeze every ounce of performance. Actix Web is fast, but with a few advanced techniques you can push it further.

Memory allocators (mimalloc/jemalloc): By default, Rust uses the system allocator, which is used for general workloads. However, in high concurrency allocators like mimalloc or jemalloc can drastically reduce fragmentation and increase allocation speed.

Switching is simple. Add the allocator as a dependency:

```
# Cargo.toml[dependencies]mimalloc = "0.1"
```

Then set it globally:

```
use mimalloc::MiMalloc;#[global_allocator]static GLOBAL:
MiMalloc = MiMalloc;
```

The small change can increase throughput in allocation workloads (e.g., JSON APIs) especially under load.

Zero-copy deserialisation (Serde): Parsing JSON can be expensive if you keep allocating memory. Serde allows you to borrow the data instead of copying it, reduces allocations and makes things faster.

Instead of owning strings:

```
use serde::Deserialize;#[derive(Deserialize)]struct User {
name: String,}
```

…use borrowed data:

```
use serde::Deserialize;#[derive(Deserialize)]struct User<'a>
{    name: &'a str,}
```

The deserialiser now points to the input buffer, without allocating new memory. This is called zero-copy deserialisation and is effective for high-throughput APIs.

SIMD acceleration (optional optimisation): For compute-heavy tasks (filtering, parsing, analytics), you can use SIMD to process multiple data points in parallel at the CPU-level. Rust offers access via crates such as *std::simd* (nightly) or libraries such as *packed_simd.*

The basic idea is:

```
use std::simd::f32x4;
let a = f32x4::from_array([1.0, 2.0, 3.0, 4.0]);
let b = f32x4::from_array([5.0, 6.0, 7.0, 8.0]);

let result = a + b; // Adds 4 numbers simultaneously
```

While SIMD isn't needed for standard CRUD APIs, it can significantly speed up workloads like analytics, scoring systems, or real-time data processing inside your API.

Testing and quality assurance

Once you implement an API, testing and quality assurance help ensure that it behaves as expected under normal and extreme conditions. Actix Web offers good tooling to make this process efficient and reliable.

Unit and integration testing: Actix Web includes utilities for testing without requiring an entire server. You can do mock requests and verify responses straight in code.

```
use actix_web::{test, App};
#[actix_web::test]
async fn test_get_items() {
    let app = test::init_service(App::new().service(list_
items)).await;
```